Knowledge Graph e Graph Machine Learning

Analisi Comparativa e Implementazione con Oracle

Sandi Russo

Università degli Studi di Messina

Scienze Informatiche

Anno Accademico 2024/2025

Relatore: docente Antonio Celesti

Agenda

Introduzione al Machine Learning e ai Grafi

Knowledge Graph e Graph Data Science

Analisi Comparativa dei Database a Grafo

Oracle Graph: La Scelta Motivata

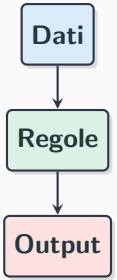
Implementazione e Setup

Conclusioni e Sviluppi Futuri

Introduzione al Machine Learning e ai Grafi

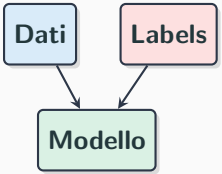
Programmazione Tradizionale vs Machine Learning

Programmazione Tradizionale



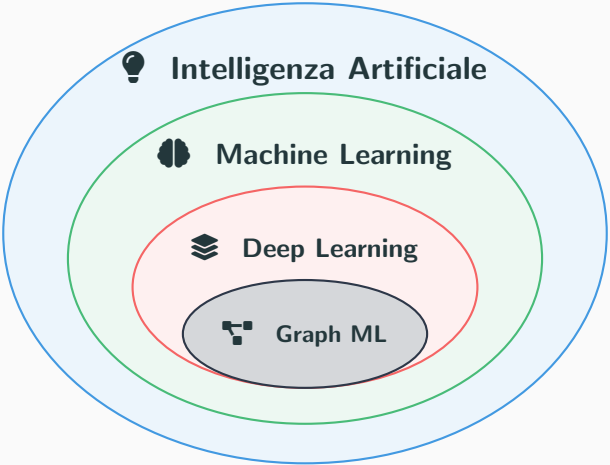
- Logica esplicita
- Regole hard-coded
- Poco flessibile

Machine Learning



- Apprendimento automatico
- Pattern discovery
- Adattivo

La Gerarchia dell'Intelligenza Artificiale



Tipologie di Apprendimento

Tipo	Descrizione
Supervised	Dati + etichette $\rightarrow$ Es: riconoscimento immagini
Unsupervised	Solo dati, senza etichette $\rightarrow$ Es: clustering utenti
Semi-supervised	Mix di dati etichettati e non etichettati

Knowledge Graph e Graph Data Science

Dal Grafo al Knowledge Graph

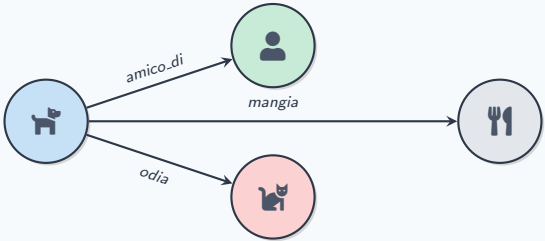
Grafo Tradizionale

- Nodi omogenei
- Archi omogenei
- Struttura semplice

Knowledge Graph

- Nodi eterogenei
- Relazioni con proprietà
- Modellazione ricca

Esempio Pratico



Graph Data Science: Capacità di Analisi

Cosa Analizza la Graph Data Science

- Evoluzione temporale di nodi e relazioni
- Identificazione dei nodi più influenti
- Interazioni e dinamiche tra gruppi
- Pattern significativi emergenti

Vantaggio Chiave

Il Graph Machine Learning **non è una black box**: la struttura dei grafi permette *interpretabilità* dei risultati.

Knowledge Graph Embedding

Obiettivo: Trasformare nodi e relazioni in vettori numerici

$$\theta : V \to \mathbb{R}^d$$

Random Walk

- Ispirato a Word2Vec
- Percorsi casuali
- Embedding *generici*

Neural Networks

- Comunicazione tra nodi
- Scambio proprietà (hop)
- Embedding *task-specific*

Applicazioni

Classificazione Link Prediction Regressione

Analisi Comparativa dei Database a Grafo

Database a Grafo: Panoramica

Caratteristica	Neo4j	Oracle	ArangoDB	Neptune
Versione gratuita	Community	Always Free	Community	Trial 30gg
GDS integrata	Sì (vasta)	Sì (PGX)	No	No
Linguaggi query	Cypher	PGQL/SQL	AQL	Gremlin
Multi-modello	No	No	Sì	No
ML Integration	Python	OML4Py	Python	SageMaker
Scalabilità	Alta	Altissima	Alta	Altissima
Curva	Media	Alta	Bassa	Alta

Neo4j: Lo Standard di Riferimento

PRO

- Community completa
- GDS vastissima
- Comunità enorme
- Docs eccellenti
- Cypher intuitivo

CONTRO

- Limitazioni free
- No multi-modello
- Costi Enterprise

Caso d'uso ideale

Progetti con budget e supporto commerciale

ArangoDB: Flessibilità Multi-Modello

PRO

- 100% open-source
- Multi-modello
- AQL flessibile
- Leggero
- Python fluido

CONTRO

- No GDS integrata
- Comunità piccola
- Librerie esterne

Caso d'uso ideale

Prototipazione rapida e flessibilità

Amazon Neptune: Cloud-Native

PRO

- Scalabilità estrema
- Fully managed
- Gremlin + SPARQL
- AWS/SageMaker

CONTRO

- Trial 30 giorni
- Costi elevati
- No GDS integrata
- Lock-in AWS

Limitazione critica

Inadatto per progetti universitari (pay-per-use)

Oracle Graph: La Scelta Motivata

Oracle Graph Server and Client: Panoramica

PRO

- Always Free Tier
- PGX potentissimo
- 60+ algoritmi
- PGQL SQL-like
- OML4Py nativo

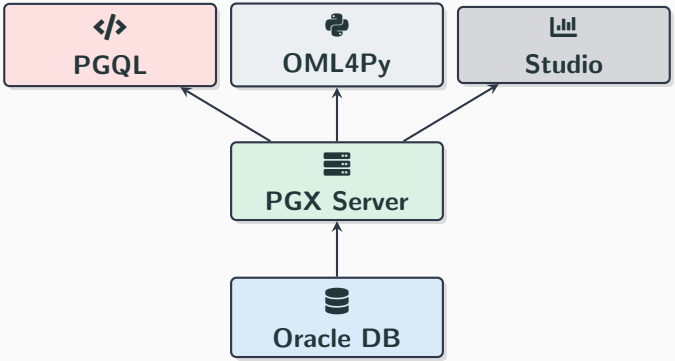
CONTRO

- Più risorse
- Curva ripida
- Meno agile
- Setup complesso

Componenti Chiave

Oracle DB | PGX Server | PGQL | OML4Py

Architettura Oracle Graph



Perché Oracle per questo Progetto?

$ Costo Zero per Ricerca/Didattica

- Always Free Tier su Oracle Cloud
- Download gratuito per uso accademico

Potenza Analitica Superiore

- PGX ottimizzato per grafi massivi
- Performance comparabili a Neo4j Enterprise

Integrazione ML Nativa

- OML4Py: ML direttamente su grafi
- PyPGX: embedding e GNN out-of-the-box

Competenze Trasferibili

- PGQL simile a SQL
- Esperienza enterprise-ready

Oracle vs Neo4j: Confronto Tecnico

Aspetto	**Neo4j Community**	**Oracle (Free)**
Costo	Gratis (limitato)	Gratis (completo)
Algoritmi GDS	65+ (50 Enterprise)	60+ (tutti)
Query	Cypher	PGQL (SQL-like)
ML	Python (esterno)	OML4Py (nativo)
Performance	Grafi medi	Grafi massivi
Deploy	Docker/Cloud	Docker/OCI
Docs	Vastissima	Ottima
Curva	Media	Alta
Vantaggio	Ecosistema	**Potenza**

Oracle vs ArangoDB: Confronto Tecnico

Aspetto	**ArangoDB**	**Oracle**
Multi-Modello	Sì (docs+grafi)	No (solo grafi)
GDS	No	Sì (PGX)
Peso	~500MB	~2GB
Query	AQL	PGQL
Algoritmi	0 (Python)	60+ (nativi)
Embedding	Esterni	PyPGX
Setup	Rapido	Complesso
Community	Crescente	Enterprise
Vantaggio	Agilità	**Analisi**

Implementazione e Setup

Setup Oracle: Docker Quick Start

Passo 1: Pull immagine

```
docker pull container-registry.oracle.com/database/graph-quickstart:latest
```

Passo 2: Avvio container

```
docker run -d --name oracle-graph-ml \
  -p 1521:1521 -p 7007:7007 \
  -e ORACLE_PWD=Password234 \
  container-registry.oracle.com/database/graph-quickstart:latest
```

Porte Esposte

1521: Oracle Database **7007**: Graph Studio

Connessione al Database

Credenziali:

```
# Graph Studio (https://localhost:7007/ui)
Username: GRAPHUSER
Password: Password234

# SQL Developer
Username: SYSTEM
Password: Password234
Hostname: localhost
Port: 1521
Service Name: FREEPDB1
```

Librerie Python:

```
pip install oracledb oml4py pypgx
```

Conclusioni e Sviluppi Futuri

Sintesi della Scelta

Perché Oracle Graph?

- **Gratuità**: Always Free Tier
- **Potenza**: PGX supera Neo4j Community
- **Completezza**: ML nativo
- **Formazione**: Esperienza enterprise

Trade-off Accettato

Complessità iniziale compensata da:

- Capacità analitiche superiori
- Nessun costo (vs Neptune)
- GDS integrata (vs ArangoDB)
- Funzionalità complete (vs Neo4j)

Limitazioni e Considerazioni

Punti di attenzione:

Risorse: 4GB RAM vs 2GB

Setup: DB + PGX separati

Docs: Meno beginner-friendly

Community: Più enterprise

Alternative:

- **Neo4j**: Ecosistema maturo
- **ArangoDB**: Setup minimale
- **Neptune**: Ambiente AWS

Sviluppi Futuri

Estensioni possibili:

Benchmark

- Performance su dataset reali
- Analisi tempi/memoria

ML Avanzato

- GNN con PyPGX
- Link prediction complessa

Web Semantico

- Import RDF/OWL
- Reasoning ontologie

Produzione

- Deploy su OCI
- API REST

Risorse e Riferimenti

Oracle:

- Property Graph: `docs.oracle.com/.../property-graph/`
- OML4Py: `docs.oracle.com/.../oml4py/`
- Graph Studio: `docs.oracle.com/.../graph-studio...`

Alternative:

- Neo4j: `graphacademy.neo4j.com`
- ArangoDB: `arangodb.com/docs/`
- Neptune: `aws.amazon.com/neptune/`

ML:

- PyTorch Geometric: `pytorch-geometric.readthedocs.io`
- DGL: `dgl.ai`

Grazie per l'attenzione!